



Amrita School of AI
COIMBATORE CAMPUS

AGENTIC AI, HANDS ON

Agentic AI, a Working Guide

Six patterns for building systems that decide what to do next

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

PROGRAMME

Faculty Development Programme

FRAMEWORK

LangChain 1.x, LangGraph 1.x

OFFERING

One-day workshop 2026-27

COMPANION TO

the six hands-on blocks

MODEL

any OpenAI-compatible server

How to read this

This guide follows the six hands-on blocks in order. Each section explains one pattern, shows the code that implements it, and then says where it goes wrong, because the failure modes are more useful than the happy path and much less often written down.

Every code listing here runs. They are drawn from the same exercises you build during the day, so what you read matches what you type.

The pattern catalogue follows Antonio Gulli's *Agentic Design Patterns*, cited by chapter throughout. Everything here is written independently; the citations are so you can read further, not because text has been reproduced.

Running the code, on any of the three platforms

Everything works on Linux, macOS and Windows. Only a few commands differ.

Install `uv` once. It also installs Python, so there is nothing to set up first.

On **Linux and macOS**:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

On **Windows**, in PowerShell rather than the old Command Prompt:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close the terminal and open a new one afterwards, on every platform. Then:

	LINUX AND MACOS	WINDOWS (POWERSHELL)
Build the environment	<code>uv sync</code>	<code>uv sync</code>
Check your work	<code>./selfcheck ex01</code>	<code>uv run python selfcheck ex01</code>
Run a demo	<code>uv run python -m exercises.ex01_tool_use.demo</code>	the same
Set a variable	<code>export FDP_CHAT_URL=...</code>	<code>\$env:FDP_CHAT_URL="..."</code>

PITFALL

`./selfcheck` on Windows. PowerShell has no concept of the `#!` line that tells Linux and macOS which interpreter to run a script with, so `./selfcheck` fails with an unhelpful message. `uv run python selfcheck` does exactly the same thing. If PowerShell refuses to run scripts at all, `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` clears it for that window.

NOTE

Anywhere this guide shows a bare command with no platform label, it is identical on all three. Notice that `uv run python -m exercises.ex01_tool_use.demo` uses dots and has no `.py` on the end: it names a Python *module* rather than a file path, which is precisely why it needs no per-platform variant.

1. What an agent actually is

1.1 A model does exactly one thing

Strip away the chat interface and a language model has a very small API. You give it a list of messages. It gives you back one message.

```
from langchain_core.messages import HumanMessage, SystemMessage

reply = model.invoke([
    SystemMessage(content="You are a maths tutor."),
    HumanMessage(content="What is 17 x 24?"),
])

reply.content          # '17 x 24 = 408'
```

That is the whole interface. Note what it is not: it is not a string in and a string out. It is a *list of messages* in, and the model has no memory of anything not in that list. Every pattern in this guide is a strategy for deciding what goes into it.

DEFINITION

Message roles. A `SystemMessage` establishes who the model is being. A `HumanMessage` is the user's turn. An `AIMessage` is the model's own turn. A fourth, `ToolMessage`, carries the result of a function the model asked to run, and that fourth one is where agents begin.

1.2 The system message is doing more work than it looks

The cheapest specialisation available to you is a sentence. The same model, given the same question, behaves differently depending on who you tell it to be.

```
def ask_as(model, role: str, question: str) -> str:
    reply = model.invoke([
        SystemMessage(content=role),
        HumanMessage(content=question),
    ])
    return reply.content
```

Ask “what is an AI agent?” as *a strict examiner who answers in one line* and as *an encouraging tutor speaking to a first-year student*, and you get two genuinely different answers. This is not decoration. In section 3 you will build a router whose three specialised handlers differ by nothing else at all.

PITFALL

The system message must come **first** in the list. A system message placed after the question is, in practice, largely ignored. This is one of those rules that costs an hour to rediscover.

1.3 The actual definition

A model answers. An agent *acts*, then answers.

DEFINITION

Agent = model + tools + a loop + a stopping condition.

The model decides. The tools let it affect or observe something outside itself. The loop lets it read the result of one action and choose the next. The stopping condition is what makes it terminate, and it is the part everyone forgets.

Consider the question *how many credits are 23AID304 and 24AIM332 together?* A model alone cannot answer it: those codes are Amrita’s and the values are in our curriculum. It will refuse, or it will invent a plausible number, and the invented number looks exactly like a real one.

An agent with a `course_credits` function does this:

1. calls `course_credits("23AID304")` → 3
2. calls `course_credits("24AIM332")` → 4
3. calls `calculator("3 + 4")` → 7
4. answers

You wrote no `if` statements. You did not specify that sequence anywhere. You described what was available, and the model worked out the procedure. Change the question and the procedure changes.

COMMON MISCONCEPTION

"So an agent is just a model with extra API calls." The difference is who chooses the sequence. In ordinary software you write the control flow and the model fills in text. In an agent the model writes the control flow at run time, and you constrain it. That is why the stopping condition matters so much: you are no longer the one guaranteeing termination.

2. Tool Use

Gulli, chapter 5. Block 1 of the workshop.

2.1 The loop

This is the entire pattern, and it is smaller than its reputation:

```
send the message list to the model
  did the reply ask for tools?
    no -> that is the final answer, stop
    yes -> run each requested tool
            append each result as a ToolMessage
            go back to the top
```

Three mechanisms make it work.

The model is told what exists. `bind_tools` sends the model a description of each function: its name, its arguments, and its docstring. Without this the model has no idea it can call anything.

The reply may contain requests instead of text. When the model wants a function run, `reply.tool_calls` is a non-empty list and `reply.content` is usually empty.

Results go back as `ToolMessage`, carrying the id. Each request has an `id`; the result must carry the same one.

2.2 The implementation

```
def run_agent(model, question, tools, max_steps=6):
    by_name = {t.name: t for t in tools}
    bound = model.bind_tools(tools)

    messages = [SystemMessage(content=SYSTEM),
                 HumanMessage(content=question)]

    for _ in range(max_steps):
        reply = bound.invoke(messages)
        messages.append(reply)

        if not reply.tool_calls:          # the stopping condition
            return reply.content

        for call in reply.tool_calls:    # possibly several at once
            tool = by_name.get(call["name"])
            if tool is None:
                result = f"There is no tool called {call['name']!r}."
            else:
                try:
                    result = tool.invoke(call["args"])
                except Exception as e:
                    result = f"{call['name']} failed: {e}"

            messages.append(ToolMessage(content=str(result),
                                         tool_call_id=call["id"]))

    raise StepBudgetExceeded(...)
```

Read the two error branches. A tool that raises should not kill the agent, and a hallucinated tool name should not raise `KeyError`. In both cases you report the problem back as a `ToolMessage` and let the model recover, which it usually does. An agent that crashes on its own mistake is far more brittle than one that is told about it.

A real trace from the School's server, for the question *how many credits do 23AID304 and 24AIM332 come to together?*

```
step 1: course_credits({'code': '23AID304'}) -> 23AID304 carries 3 credits.
step 1: course_credits({'code': '24AIM332'}) -> 24AIM332 carries 4 credits.
step 2: calculator({'expression': '3 + 4'}) -> 7
step 3: no tools wanted – this is the answer
```

A: The total number of credits for 23AID304 and 24AIM332 is 7.

WORKED EXAMPLE

Both lookups came back in **one** reply — note that they are both step 1. That is why the code loops over `reply.tool_calls` rather than taking the first element. The model recognised two independent lookups, asked for both at once, then used the results in a third call to arrive at the answer. Nobody scripted that sequence; it was chosen at run time from the question and the tool descriptions.

2.3 The docstring is the interface

The model never sees your function body. It sees the name, the argument names and types, and the docstring, and decides from those alone.

```
@tool
def course_credits(code: str) -> str:
    """Look up how many credits an Amrita course carries, given its course code.

    Use this whenever a question mentions a course code such as 23AID304.
    Returns the credit count, or says so if the code is not in the catalogue.
    """
```

That docstring is not documentation, it is a prompt. Make it vague and the model calls the function at the wrong times or not at all. This is worth an experiment: degrade one docstring, re-run the demo, and watch the agent's behaviour change without a line of logic changing.

PITFALL

Forgetting `tool_call_id`, or inventing a new one. The model matches results to requests by that id, and it invents nothing. Get it wrong when two tools were requested at once and the model reads the answer to a question it never asked. The symptom is a confidently wrong final answer with no error anywhere.

PITFALL

A loop with no way out. If your code never checks for an empty `tool_calls`, or the model simply keeps asking, the loop does not terminate. Nothing raises. On a metered API this is the failure that shows up on the invoice rather than in the tests. Every agent needs a step budget.

BEYOND THE SYLLABUS

Tools are a security boundary. A tool is a direct route from model-generated text to your machine. The calculator in block 1 parses the expression into a syntax tree and evaluates only arithmetic nodes, rather than calling `eval`, precisely for this reason. The model is repeating patterns it read on the internet; treat its output as untrusted input, because that is what it is. Gulli's chapter 18 is about this class of problem.

3. Prompt Chaining and Routing

Gulli, chapters 1 and 2. Block 2 of the workshop.

3.1 Chaining: one instruction per call

Ask a model to “read this and give me a headline” and you have asked for two jobs at once: work out what matters, then compress it. It does both, and does both slightly worse than it would do either alone.

```
text → [extract the facts] → facts → [headline these facts] → headline
```

```
def chain(model, text):
    facts = model.invoke([
        SystemMessage(content=EXTRACT),
        HumanMessage(content=text),          # the original
    ]).content

    headline = model.invoke([
        SystemMessage(content=HEADLINE),
        HumanMessage(content=facts),         # stage one's OUTPUT
    ]).content

    return facts, headline
```

The defining property is on the second call: it consumes `facts`, not `text`. If stage two re-reads the original input you have written two independent calls, not a chain, and you have paid twice for nothing.

The real benefit is the middle value. When the headline is wrong you can look at `facts` and see immediately whether the extraction failed or the compression did. A single call gives you no such handle: it is wrong, and you have nowhere to look. That intermediate is also something you can log, cache, or hand to a human to approve.

COMMON MISCONCEPTION

“Chaining is just splitting the prompt up, so it must be slower and dearer.” It is slower and dearer. You are buying reliability and inspectability with latency and tokens. Whether that is a good trade depends entirely on what you are building, and deciding is the engineering.

3.2 Routing: classify, then dispatch

One assistant that handles everything is mediocre at all of it. Ask first what *kind* of question this is, then hand it to a handler built for that kind.

```
ROUTES = {
    "maths": "You are a maths tutor. Solve it and show one line of working.",
    "policy": "You are an academic office clerk. Answer plainly...",
    "other": "You are a helpful assistant. Answer briefly.",
}
```

Notice that the three handlers differ by **nothing but their system message**. Same model, same code path, three behaviours, at essentially no cost. This is why section 1.2 spent time on system messages.

```
def pick_route(model, question):
    reply = model.invoke([
        SystemMessage(content=ROUTER),
        HumanMessage(content=question),
    ]).content

    label = reply.strip().strip(".!,:;\n'").lower()
    return label if label in ROUTES else "other"
```

3.3 The two lines that matter

Those last two lines are the whole lesson of this section.

You will ask the model for exactly one word. You will receive "Maths.", or " MATHS ", or, once in fifty, "I think this is about chemistry". A router written as ROUTES[reply] raises KeyError in front of a room full of people.

So: normalise, and fall back. Neither has anything to do with AI. It is ordinary defensive coding against an input you do not control, and the only unusual thing is that people forget to do it because the input came from a model rather than a user.

PITFALL

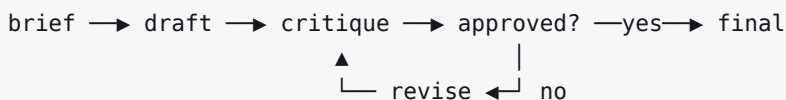
Routing to a handler that is not actually different. If every route ends up using the same generic system message, you have paid for a classification call and changed nothing. Check that your handlers really diverge.

4. Reflection

Gulli, chapter 4. Block 3 of the workshop.

4.1 The model marks its own work

A model's first answer is a first draft. Ask the *same model* to review that draft against the original brief and it finds real faults: vagueness, a missing example, a part of the question left unanswered. Feed the critique back and ask for a revision, and the second draft is usually better.



4.2 Why this is not circular

The obvious objection: if the model could write the better version, why did it not write it first?

Because writing and judging are different tasks, and a model doing one is not doing the other. Generating text, it is producing a continuation. Handed a finished text and asked what is wrong with it, it attends to different features entirely. The second task is genuinely easier — it is easier for people too, which is why we have editors.

```
def reflect(model, brief, max_rounds=3):
    draft = _say(model, DRAFT, brief)
    result = Reflection(final=draft, rounds=0, drafts=[draft])

    for _ in range(max_rounds):
        critique = _say(model, CRITIC,
                        f"BRIEF:\n{brief}\n\nTEXT:\n{draft}")
        result.critiques.append(critique)

        if APPROVAL in critique.upper():          # not == , see below
            result.approved = True
            result.final = draft
            result.rounds += 1
            return result

        draft = _say(model, REVISE,
                    f"BRIEF:\n{brief}\n\nCURRENT TEXT:\n{draft}"
                    f"\n\nCRITIQUE:\n{critique}")
        result.drafts.append(draft)
        result.final = draft                      # keep the best so far
        result.rounds += 1

    return result
```

Three details are load-bearing.

The critic sees the brief as well as the draft. It cannot judge whether text meets a brief it has never been shown. This omission is common and produces critiques that are fluent and useless.

The reviser sees the critique. Otherwise it rewrites at random and nothing improves, while costing exactly as much.

`APPROVAL in critique.upper()`, not `critique == APPROVAL`. You will instruct the critic to reply with one word. It will reply `APPROVED.` or `This is APPROVED - well done!`. An equality test silently fails to detect approval, so your loop runs the full three rounds every single time and you pay triple for a result you already had after round one.

4.3 Deciding whether to spend it

Reflection turns one call into three or five. Sometimes that buys a much better answer and sometimes it buys almost nothing, and which one you are in depends on the task.

The way to find out is to keep the drafts and read them, which is why the implementation returns every draft and every critique rather than only the final text. If draft one and draft three are nearly identical for your use case, you have your answer.

PITFALL

A critic that never approves. Some briefs are impossible to satisfy, and some critics are simply harsh. Without `max_rounds` the loop does not terminate. When the rounds run out, return the most revised draft rather than raising: it is the best version you have, and throwing it away helps nobody.

5. Knowledge Retrieval

Gulli, chapter 14. Block 4 of the workshop.

5.1 When a tool is not enough

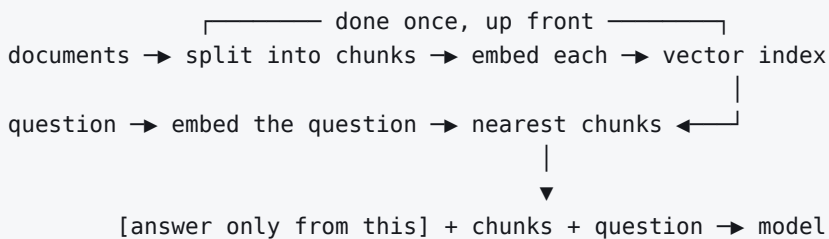
Block 1 solved “the model does not know our credit values” with a function. That works when the knowledge is a lookup with a clean key.

It does not work for a 74-page curriculum document. There is no `get_answer()` for *what does the fourth-semester project involve*. The knowledge is prose and the question could be about any part of it.

The obvious fix, pasting the whole document into the prompt, fails on both cost and quality: you pay for every token on every question, and a model given seventy pages attends worse than one given the relevant two paragraphs.

5.2 The move

Do not give the model the document. Give it *the paragraphs of the document that relate to this question*.



DEFINITION

Embedding. Turning text into a list of numbers positioned so that text about similar things lands nearby. Once both your chunks and your question are points in the same space, “find relevant passages” becomes “find nearby points”, which computers are extremely good at.

```

def split(docs):
    splitter = RecursiveCharacterTextSplitter(chunk_size=800,
                                              chunk_overlap=120)
    return splitter.split_documents(docs)

def build_index(chunks, embeddings):
    return FAISS.from_documents(chunks, embeddings)

def answer(model, index, question, k=4):
    chunks = index.similarity_search(question, k=k)
    context = "\n\n".join(
        f"[{c.metadata['source']}] \n {c.page_content}" for c in chunks
    )

    reply = model.invoke([
        SystemMessage(content=ANSWER),
        HumanMessage(content=f"CONTEXT: \n {context} \n\n QUESTION: {question}"),
    ])
    return reply.content, chunks

```

The function returns the chunks alongside the answer. An answer you cannot trace to a source is not much better than a guess, and being able to read what was retrieved is how you develop judgement about whether the system is working.

5.3 The grounding prompt

```

ANSWER = (
    "Answer the question using only the context provided.\n"
    "If the context does not contain the answer, say exactly: "\n"
    "I cannot find that in the documents.\n"
    "Do not use anything you know outside the context. Cite the source file "\n"
    "name for each fact you use."
)

```

Without this, retrieval is decorative: the model reads your context, ignores it, and answers from training data anyway. The explicit refusal instruction matters most. A system that says *I cannot find that in the documents* is far more useful than one that produces a confident, plausible, invented answer, because the second kind cannot be distinguished from a correct one by looking at it.

PITFALL

Retrieving, then not using it. The commonest RAG bug by a wide margin. You build the index, retrieve four perfect passages, and send the model the bare question. It answers from memory, sounds completely confident, and nothing in the output tells you the retrieval was thrown away. If your answers do not change when you break the retriever, you never had RAG.

5.4 Chunk size, and why nobody can tell you the right value

800 characters with 120 of overlap, in this material. The numbers are a starting point, not a law.

Too large, and each chunk covers several topics, so retrieving it hands the model noise along with the answer. Too small, and a chunk stops containing a complete thought: you retrieve “*the programme requires*” and the number lives in the next chunk.

The overlap exists because the cut points are arbitrary. Without it, a fact sitting exactly on a boundary is halved and found by neither piece.

The only way to choose well is to try values on *your* documents and read what comes back. Change it in `split()`, re-run the demo, and watch the answers move.

BEYOND THE SYLLABUS

Retrieval does not have to mean embeddings. Scoring passages by how many of the question’s content words they contain is crude, costs nothing, needs no model, and works surprisingly often. Block 5 uses exactly that. Embeddings win when the question and the passage use different words for the same thing, which is often but not always. Starting with keyword scoring and moving to embeddings when you can show it helps is a defensible engineering order.

5.5 Boilerplate, and the first thing to fix in a disappointing RAG system

Run block 4’s demo and one of the questions fails in a specific, instructive way. Ask which semester the project work falls in, and what comes back is four near-identical chunks reading *Evaluation Pattern - 70:30, Midterm Exam - 20%, Lab Assignments - 25%...*

A curriculum document repeats that block under every single course. Dozens of chunks are therefore almost the same text, they all score similarly against a question mentioning “project”, and between them they crowd out the one passage that actually answers it.

This is not a model failure and no amount of prompt work fixes it. Real corpora are full of it: page headers, footers, standard disclaimers, repeated tables, navigation text scraped along with content.

PITFALL

Near-duplicate chunks dominate retrieval. If a phrase appears two hundred times in your corpus, it will fill your top-k for any question that brushes against it. Deduplicating near-identical chunks before indexing is usually the single highest-value change to a RAG system that underperforms, and it is ordinary data cleaning rather than anything to do with AI.

Worth knowing about, in rough order of effort: drop chunks whose text is a near-duplicate of one already indexed; strip repeated headers and footers before splitting; retrieve more than you need and then filter for diversity; or weight terms by how rare they are across the corpus, so a word appearing in every chunk counts for little.

None of that is in this workshop. The point of showing you the failure is that you will recognise it, and know it is a cleaning problem rather than a reason to reach for a bigger model.

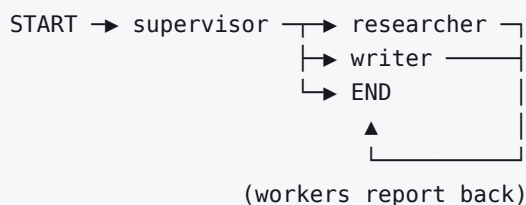
6. Multi-Agent systems, and knowing when to stop

Gulli, chapters 7 and 11. Block 5 of the workshop.

6.1 Why more than one

A single agent with six tools and a long system prompt degrades. The prompt grows to cover every case, the instructions begin to contradict one another, and behaviour becomes hard to predict and harder to change without breaking something else.

Splitting the work into agents with narrow jobs is the same move as splitting a long function. Each part is small enough to reason about, and you can change one without disturbing the others.



A **researcher** that only finds facts. A **writer** that only writes from facts it is given. A **supervisor** that decides which acts next.

Both workers return to the supervisor. That is what makes this a supervised team rather than a fixed pipeline: after each piece of work someone decides again, so the same graph handles “needs three rounds of research” and “answerable immediately” without an `if` anywhere.

6.2 State, as a graph

```
class State(TypedDict):
    question: str
    notes: Annotated[list[str], lambda a, b: a + b] # accumulates
    draft: str
    steps: int
    next: str
```

The annotation on `notes` says how two updates combine: by concatenation. Each research round therefore *adds* to what is known rather than replacing it. The other fields overwrite, which is what you want for a draft.

```
def build_graph(model):
    graph = StateGraph(State)

    graph.add_node("supervisor", lambda s: supervisor_node(s, model))
    graph.add_node("research", lambda s: research_node(s, model))
    graph.add_node("write", lambda s: writer_node(s, model))

    graph.add_edge(START, "supervisor")
    graph.add_conditional_edges(
        "supervisor", route,
        {"research": "research", "write": "write", END: END},
    )
    graph.add_edge("research", "supervisor")
    graph.add_edge("write", "supervisor")

    return graph.compile()
```

6.3 The part that is really the lesson

Nothing in this system stops on its own.

The supervisor can keep saying RESEARCH. The researcher will keep researching. No exception is raised, no test fails, nothing looks broken. The system simply runs, and on a metered endpoint it runs while you are at lunch.

So the state carries a step counter and the router checks it:

```
def route(state):
    if state["steps"] >= MAX_STEPS:      # FIRST
        return END
    if state["next"] == "RESEARCH":
        return "research"
    if state["next"] == "WRITE":
        return "write"
    return END
```

DEFINITION

Goal monitoring. An explicit, external condition under which the system stops, which does not depend on the model choosing to stop. A step budget is the simplest form. Others: a wall-clock deadline, a token budget, or a test that the goal is objectively met.

The order is not stylistic. Checking the decision before the budget grants one extra round every single time the budget binds. With a large model and a long context that is a real cost, and it is invisible in any test that only checks the final answer.

PITFALL

Trusting the model to terminate. The supervisor is a model, and models are agreeable. Asked whether more research would help, “yes” is always a defensible answer. Autonomy without a budget is not autonomy, it is an unbounded loop with good manners.

6.4 Everything else, showing up again

The capstone is not a new pattern so much as the previous five arranged together.

IN THE CAPSTONE	IS REALLY
The supervisor choosing a worker	Routing (§3.2), same normalise - and - fall - back problem
The researcher calling <code>search_corpus</code>	Tool use (§2), results kept as state rather than <code>ToolMessage</code>
Searching the curriculum	Retrieval (§5), by keyword rather than vector
The supervisor judging the draft	Reflection (§4), a critique step by another name
Research then write	Chaining (§3.1), with the sequence chosen at run time

That is the honest summary of the field at this level: a small number of simple mechanisms, composed.

7. Choosing between the patterns

The patterns are easy. Knowing which one a problem needs is the part that takes practice, and it is mostly a matter of asking what is actually missing.

THE PROBLEM	REACH FOR
It does not know a fact you can look up with a key	Tool use
It does not know something written down in prose	Retrieval
It needs to do something in the world	Tool use
One call is being asked to do two jobs	Chaining
Different kinds of question need different treatment	Routing
The first answer is nearly right but sloppy	Reflection
The prompt has grown to cover too many cases	Multi-agent
It runs and runs	Goal monitoring, and you needed it already

Two habits worth forming.

Start with the simplest thing that could work. A single call with a good system message solves more than people expect. Reach for reflection when you can show the first draft is not good enough, not because reflection is interesting.

Measure before you add. Every pattern here costs calls, latency and money. The way to know whether one earned its place is to look at the output with and without it, which means keeping the intermediate values rather than discarding them.

8. What this guide leaves out

Stated plainly, so you know the shape of the gap.

Evaluation. The most serious omission. Everything here tells you whether a pattern is *wired* correctly, which is not the same as whether the agent's answers are *good*. Measuring output quality is its own subject, needs its own data, and a second workshop could be built around it alone. Gulli's chapter 19 is the place to start.

Memory across sessions. Everything here is stateless between runs. Real assistants remember. Chapter 8.

Safety and guardrails. Touched only where it intersects tool design in §2.3. Prompt injection, in particular, becomes a live concern the moment an agent retrieves text somebody else wrote. Chapter 18.

Cost and latency in production. Batching, caching, streaming, choosing a smaller model per step. Chapter 16.

Everything about deployment. Monitoring, tracing, versioning prompts, handling a model provider changing behaviour beneath you.

9. Further reading

Antonio Gulli, *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*. Twenty-one patterns, code for each, and the source this workshop's catalogue follows.

The chapters closest to the six blocks:

CHAPTER	PATTERN	BLOCK
1	Prompt Chaining	2
2	Routing	2
4	Reflection	3
5	Tool Use	1
7	Multi-Agent	5
11	Goal Setting and Monitoring	5
14	Knowledge Retrieval	4

And the four worth reading next, none of which is covered here: **8** memory, **16** resource-aware optimisation, **18** guardrails, **19** evaluation.

The framework documentation for LangChain and LangGraph is good and changes often; prefer it to any tutorial, including this one, on questions of syntax. The patterns will outlast the syntax.